QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control 3
PRELIMINARY ROUND (Test Strategy & Planning) 3
1. Scope & Requirements Traceability 3
1.1 In-Scope Core Features 3
1.2 Out-of-Scope 4
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 5
4.1 Integration Thresholds (Merging to Main) 6
5. Test Case Specifications 7
6. AI Output & Boundary Testing 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*[e.g., UMHackathon 2026 - Team Pulau Pinang Mantap]*
Team Repo URL	https://github.com/geatsXIHaha/review2revenue
Project Board URL	*[Insert link to your Jira/Trello Projects board]*
Live Deployment URL (optional for preliminary round)	*[Insert link to the live hosted application]*

Objective:

The primary objective is to ensure that Review2Revenue reliably handles role-based registration (Diner/Vendor), securely manages database integrations (Firebase/Supabase), effectively routes AI requests with proper fallbacks, and accurately processes restaurant/review data while maintaining high performance.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via Requirement Traceability Matrix". So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **Authentication & Role Management:** Firebase Google Auth integration, Supabase profile creation, and immutable role selection (Diner vs. Vendor).
- **Restaurant Discovery (Diner):** Case-insensitive restaurant search, AI recommendation routing, and UI rendering of restaurant cards alongside chat history.
- **Business Intelligence (Vendor):** Single-restaurant lock via store_id, retrieval of internal metrics, and generation of sentiment summaries using external LLMs or the local TF-IDF model.
- **API & Fallback Logic:** /api/ask routing, chat history persistence, and the zero-dependency fallback mechanism when external LLMs (Groq/Gemini/Z.AI) are unavailable.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Out-of-Scope

- Post-registration role switching.
- Full menu editing or restaurant profile image management.
- Legacy Streamlit dashboard UI components.

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
External LLM API Timeout or Failure	*3*	*5*	*15 (High)*	*Implement automatic rollback strategy to handle such failures. [e.g. Database transaction timeout increased to 10s]*	*Trigger /api/ask while blocking external network requests to verify local model activation.*
Vendor registers for non-existent restaurant	*2*	*4*	*8(Medium)*	*Require case-insensitive ILIKE database search and selection before allowing Supabase upsert.*	*Attempt to force-submit registration form with an invalid store_id payload.*
Chat history context lost on refresh	*3*	*3*	*9(Medium)*	*Persist chat turns in the chat_messages PostgreSQL table linked to conversation_id.*	*Refresh ChatPage.jsx mid-conversation and verify history via /api/chat/history.*
Firebase UID and Supabase ID mismatch	*1*	*5*	*5 (Low)*	*Strictly map Firebase UID as the Primary Key in the Supabase users table.*	*End-to-end integration test of the registration hook.*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - ***Scope:*** *FastAPI endpoints (/api/restaurants/search) and local sentiment engine logic.*
  - ***Execution:*** *Automated via PyTest on backend code.*
  - ***Isolation:*** *PostgreSQL database calls are mocked to test pure search logic and API response schemas.*
  - ***Pass Condition:*** *Expected JSON format returned with HTTP 200; empty queries return appropriate HTTP 400 validation errors.*
- **Integration Test**
  - ***Scope:*** *Firebase Auth -> Supabase Profile Upsert -> ProtectedApp.jsx Routing.*
  - ***Execution:*** *Executed post-merge to the main branch.*
  - ***Workflow:*** *Real calls made to a staging Supabase database to ensure Foreign Key constraints (e.g., store_id) execute correctly.*
  - ***Pass Condition:*** *User is seamlessly routed from the registration screen to the main app dashboard based on their saved DB role.*
- **Test Environment (CI/CD Practice):**
  - ***Local Testing:*** *Manual testing via Vite dev server (localhost:5173) and Uvicorn (localhost:8000).*
  - ***Staging/CI:*** *GitHub Actions triggers linting and automated API tests upon Pull Requests.*

- **Regression Testing & Pass/Fail Rules:**
  - ***Execution Phase:*** *Run against the core /api/ask and registration flows on every major*

© UMHackathon 2026

Email: umhackathon@um.edu.my

release.
    - ***Pass/Fail Condition:*** *Any failure in the database fallback logic or RLS policy checks constitutes an immediate failure.*

- **Test Data Strategy:**
    - ***Manual:*** *3 mock accounts (1 Diner, 2 Vendors assigned to different store_ids).*
    - ***Automated:*** *Database seeded with synthetic metadata using data/clean_restaurants_enriched.csv and data/clean_reviews.csv.*

- **Passing Rate Threshold:**
    - *A minimum of 90% of all executed test cases must pass. For critical tests—specifically Firebase Google Authentication, Supabase profile upserts, immutable role assignments (Diner/Vendor), and database constraint validations (such as the store_id Foreign Key)—all tests (100%) must pass with no exceptions. Critical failures in these core authentication and data-integrity flows would completely break the user routing and application functionality in production.*

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error (Vite/Python)***	***0 Errors***	***Passed***
Unit Tests	***90% Passing Rate on API***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***100%***	***Passed***
Test Coverage	***Minimum 90.00%***	***100%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Zero FK Constraint Violations***	***0***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***92%***	***Passed***
Critical Bugs	***Zero Auth/Registration Bugs***	***0***	***Passed***
API Performance	***Search API Response < 300ms***	***210ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

5. Test Case Specifications

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***Happy Case:*** *Vendor Registration Flow*	*Verify a new Vendor can authenticate, search their restaurant, and save their profile to access insights.each other correctly.*	*. Click "Sign in with Google".* *2. Select "Vendor" role.* *3. Search restaurant*	*Supabase users table updates with role: vendor and correct store_id. UI routes to Vendor Dashboard displaying correct restaurant data.*	*Profile saved successfully. Dashboard loads specific store_id data via /api/restaurants /by-store-id. and delivered the correct order in time.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			name. *4. Select from dropdown.* *5. Confirm registration.*		*Status: Passed*
TC-02	***Specific Case (Negative):*** *Missing Restaurant*	*Verify system handles vendors searching for a restaurant not in the PostgreSQL database.*	*1. Authenticate via Google.* *2. Select "Vendor".* *3. Type random unregistered string.* *4. Attempt to proceed.*	*Dropdown shows "No results". Submission is blocked. UI displays dismissible error banner preventing FK constraint violation.*	*Form submission blocked. Error banner displayed. App remains stable.* *Status: Passed*
TC-03	***NFR (Performance):*** *Search Debounce*	*Verify the case-insensitive search (ILIKE) handles rapid keystrokes without overloading the DB.*	*1. Open Network tab.* *2. Rapidly type a 10-character restaurant name in the search bar.*	*Frontend implements 300ms debounce. Only 1-2 API calls are dispatched to /api/restaurants /search.*	*Only 1 API request fired after typing paused. Response received in 205ms.* *Status: Passed*

6. AI Output & Boundary Testing

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document at least 3 Prompts/response test pairs. For each of them, define the acceptance criteria—what a correct, acceptable, or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Diner: "Recommend a place for late-night spicy food."	The response includes AI rationale text + structured JSON for the UI to render restaurant cards with matching late-night tags.	Output returned 3 recommendations with appropriate DB-backed cards mapped accurately to Google Places synthetic data.	Pass
AI-02	Vendor: "Summarize my latest reviews." (External API Key Disabled)	The system must intercept the failure, trigger the trained_model fallback, and return local sentiment with model_confidence .	Returned keyword fallback summary. model_confidence appended to UI. No 500 server errors encountered.	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	2000 characters for user prompt; max 50 objects in external_reviews array.
Input used while testing	5000-character prompt containing 150 simulated review payloads.
Expected Behavior	The backend truncates the prompt to the limit and processes only the first 50 reviews to prevent token exhaustion.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Actual Behavior	System truncated text properly. API responded within 1.5s with a partial summary.
Status	Passed

6.3. Adversarial/Edge Prompt Test

In this section, test a minimum of 1 with one prompt that is ambiguous and malformed—document how GLM does respond and how your system is going to handle it.

Test: A Diner inputs SQL injection string ('; DROP TABLE users; --) into the Chat prompt.

System Handling: The FastAPI backend utilizes Pydantic schemas and parameterized SQLAlchemy/PostgreSQL queries. The prompt is passed purely as a string literal to the LLM and database, preventing any execution. The LLM simply responds with confusion regarding the food recommendation request.

6.4. Hallucinating Handling

This section is defined for you to describe how your system does detect or mitigate hallucinating responses.

Detection/Mitigation: The system utilizes a Retrieval-Augmented Generation (RAG) approach. The AI is not allowed to generate restaurant data from its pre-training. It is strictly injected with context from PostgreSQL (store_id, name, avg_rating, operating_hours). If a user asks for a restaurant not in the DB, the AI is instructed to state it cannot find the data locally, rather than fabricating a menu or rating.

Important Note:

Using Agile principles, these tests will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my